Architecture 1001: x86-64 Assembly
Writing Assembly: GCC

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

VisualStudio Intrinsics

- A subset of assembly instructions can be inserted into C code by using VS compiler intrinsics
- These look like C functions calls, but the compiler substitutes them with literal assembly instructions
- Examples:
  - __stosX (rep stos)
  - __movsX (rep movs)
  - _rotrX (ror)
- Many more (mostly relevant to Arch2001):
  - http://msdn.microsoft.com/en-us/library/hh977022.aspx

VisualStudio Assembly

- Visual Studio supports inline assembler for 32-bit code, but not x86-64
- Hand-written assembly must be written in MASM (Microsoft Assembler syntax) in a separate .asm file, with exported function names, and then called from your C code and linked to by the linker
- How to create a basic project is shown on the site, but one has been provided already in the "scratchpad_asm" project

Writing Bytes to Get Asm

- You can emit bytes directly in the instruction stream in MASM by just writing "db XXh" and then the byte you want to emit. E.g.

  db 04h

  db 01h

  Or

  db 04h, 01h

- Would create the instruction "add al, 1"

ADD—Add

Opcode	Instruction	Op/ En	64-bit Mode
04 *ib*	ADD AL, *imm8*	I	Valid

Misc MASM Syntax Issue

- MASM doesn't like immediates that start with an alphabet character. They must be preceded by a zero
  - "mov rax, E09966C1A977E177h" yields a compile error
  - "mov rax, 0E09966C1A977E177h" is OK
- This also applies to raw bytes
  - "DB C3h" errors out
  - "DB 0C3h" is OK

Lab: Create a Project With MASM

- This will show how to get Visual Studio to play nice with MASM files

Lab: RTFM && WTFI!

- Read The Fun Manual and Write The Fun Instructions!
- Write raw bytes to create the following assembly sequence:

```
mov eax, 0xAABBCCDD
sahf ; What's that? IDK, RTFM!
jz mylabel
and eax, 0x31337
mylabel:
ret
```